



COLEÇÃO ACADÊMICA · DATAGEO ACADEMY

Deep Learning Geoespacial com CNNs

Inteligência Artificial e Data Science

12 capítulos próprios · Código executável

Tabelas · Exercícios comentados

Autoria institucional: DataGeo Academy

Conhecimento que gera decisões | Edição 3.0 | 2026

Folha de rosto e controle documental

Deep Learning Geoespacial com CNNs

Autoria e edição institucional: DataGeo Academy.

CAMPO	REGISTRO
Edição	3.0 / 2026 / PDF pesquisável / 42 páginas
Organização	12 capítulos exclusivos deste tema; teoria, programação e aplicação
Dados	Sintéticos e identificados; não representam medições de um município
Bibliotecas	torch>=2.8,<3, numpy>=2,<3
Fontes editoriais	scripts_automacao/livros_ia; fonte completa anexada ao PDF
Verificação	12 laboratórios executados; asserções e saídas incorporadas
Caso integrador	Treinar uma CNN pequena para segmentação binária em patches sintéticos; separar cenas inteiras, mascarar nodata, calcular IoU por cena e reconstruir uma imagem por janelas sobrepostas. Entregar pesos, esquema de bandas, parâmetros de normalização e um relatório que diferencie o teste didático de uma validação com sensores reais.

O conteúdo deste volume foi escrito para sua sequência temática. A autoria é institucional; não há atribuição a pesquisadores ou revisores externos. A composição usa a identidade visual da Academy, mas os capítulos, exemplos e exercícios são independentes dos outros três volumes desta edição.

Os códigos são completos e executáveis em ambiente com as dependências indicadas. Os exercícios pedem alterações adicionais, e seus comentários orientam a solução. A validação automatizada verifica execução e propriedades declaradas; não constitui revisão científica externa nem certificação de resultados de campo.

REVISÃO	ESCOPO
3.0	Substituição do volume provisório por conteúdo específico e APIs reais.
Adaptação	Registrar autor da mudança, fonte, período, parâmetros e evidências.

Sumário técnico progressivo

CAP.	TEMA	PÁGINA
1	Tensores multiespectrais e contrato NCHW	5
2	Normalização por banda e máscara de nodata	8
3	Convolução, padding e campo receptivo	11
4	Pooling e classificação de patches	14
5	Encoder-decoder e conexão de resolução	17
6	Treinamento, gradientes e otimizador	20
7	Segmentação binária e perdas estáveis	23
8	Augmentação geométrica conjunta	26
9	Métricas de segmentação e classe rara	29
10	Validação por cena e modos train e eval	32
11	Inferência por janelas e reconstrução sem costuras	35
12	Checkpoint, esquema e reprodução de inferência	38

Cada capítulo possui três partes: fundamentos e uma tabela de decisões; laboratório integral com resultado calculado; exercícios autorais com comentários e uma aplicação ampliada. Os marcadores do PDF levam diretamente ao início de cada capítulo.

Os anexos incluem doze scripts Python independentes, fonte editorial em JSON, saídas de referência e requisitos do ambiente. Use um leitor com suporte a anexos ou o extrator fornecido no projeto. Execute os exemplos após ler o contrato e conferir as dependências.

COMPLEMENTO	PÁGINA
Introdução e percurso de estudo	4
Referências e ambiente	41
Projeto integrador e rubrica	42

Introdução e percurso de estudo

Uma CNN geoespacial trabalha com imagens cuja estrutura tem significado: bandas possuem unidades e respostas espectrais diferentes; pixels têm resolução, posição e validade; cenas pertencem a datas e territórios. Reduzir esse conjunto a um tensor sem registrar seus metadados destrói parte da informação necessária para interpretar o resultado. Este livro conecta operações de convolução a essas condições de uso.

Os laboratórios executam PyTorch em CPU sobre pequenos patches sintéticos. Não dependem de pesos baixados, GPU ou cenas comerciais. Os tensores artificiais permitem conferir formatos, máscaras, perdas e gradientes. O estudante aprende a construir uma rede e a treinar, avaliar e executar inferência de verdade; a acurácia obtida nesse material não estima desempenho de um satélite real.

A sequência começa pelo contrato NCHW e pela normalização por banda. Depois analisa convolução, pooling, classificação e segmentação. O núcleo de treinamento inclui perdas numericamente estáveis, propagação de gradiente, otimizador e modos train/eval. A etapa final aborda augmentação conjunta, validação por cena, inferência em janelas e persistência de pesos.

O caso integrador é a segmentação binária de superfície construída em mosaicos. Os exemplos são deliberadamente pequenos para que um tensor possa ser inspecionado e um lote executado em segundos. A versão de referência é PyTorch 2.8 CPU; threads e seeds são controlados nos exercícios que envolvem otimização. Resultados exatos de ponto flutuante podem variar entre plataformas, por isso asserções protegem propriedades matemáticas e formatos.

Ao final, a entrega esperada inclui especificação das bandas, normalização aprendida no treino, divisão por cenas, arquitetura, checkpoint, inferência sem gradiente, máscara de nodata e métricas por região. Uma rede mais profunda não compensa rótulos desalinhados, vazamento entre patches ou ausência de metadados espaciais. Esses fatores têm o mesmo peso de revisão que a escolha do otimizador.

CAPÍTULO 01 · TEORIA E MÉTODO

1. Tensores multiespectrais e contrato NCHW

1.1 Fundamentos

PyTorch utiliza normalmente lotes de imagens no formato NCHW: número de exemplos, canais, altura e largura. Dados de leitores raster podem chegar como CHW, enquanto bibliotecas de imagem frequentemente produzem HWC. Uma troca silenciosa de eixos pode manter o código executável e mudar completamente o significado de canais e vizinhança.

Cada canal precisa de identificação de banda, unidade, escala e máscara de validade. Um tensor float32 não informa sozinho se representa radiância, reflectância ou número digital. Também não guarda CRS nem transformada afim. Mantenha esses metadados em um contrato associado ao tensor e restaure-os na saída. Ordem de bandas diferente daquela usada no treino invalida a inferência.

O laboratório converte uma pequena imagem HWC de três bandas para CHW e acrescenta a dimensão de lote. O teste de posição confirma que o valor de um pixel não foi alterado pela permutação. contiguous garante disposição contígua quando necessária, mas não substitui a escolha correta dos eixos. Antes de treinar, confira forma, dtype, finitude e semântica de cada canal.

Hipóteses e aprofundamento técnico

Um tensor possui shape, dtype, device e strides. A permutação de eixos altera strides e pode produzir uma view não contígua; certas operações posteriores demandam contiguous ou reshape compatível. O custo em memória cresce com $N \times C \times H \times W$ e bytes por elemento. Por exemplo, um lote de dezesseis patches de doze bandas e 256×256 em float32 ocupa cerca de 48 MiB apenas nas entradas; ativações e gradientes elevam o total. Planeje lote e janela conforme memória, sem diminuir resolução silenciosamente. O contrato de bandas deve acompanhar também escala radiométrica e máscara.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
N	1 imagem	Dimensão do lote
C	3 bandas	Ordem deve permanecer estável
H e W	8×8 pixels	Suporte espacial
Metadados	CRS e transformada fora do tensor	Restaurar na saída

Confirme formato, valor de referência e identificação das bandas. Número de canais correto não demonstra ordem correta.

CAPÍTULO 01 · PROGRAMAÇÃO

1.2 Laboratório executável

Arquivo incorporado: capitulo_01.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
hwc=torch.arange(8*8*3, dtype=torch.float32).reshape(8,8,3)
chw=hwc.permute(2,0,1).contiguous()
lote=chw.unsqueeze(0)
assert lote.shape==(1,3,8,8)
assert lote[0,2,4,5]==hwc[4,5,2]
assert lote.dtype==torch.float32 and torch.isfinite(lote).all()
RESULTADO={'formato':list(lote.shape),
            'pixel_banda_2':float(lote[0,2,4,5]), 'contiguo':lote.is_contiguous() }
print(RESULTADO)
```

Resultado de referência calculado

```
{"contiguo": true, "formato": [1, 3, 8, 8], "pixel_banda_2": 113.0}
```

Leitura do código

arange permite identificar numericamente cada posição antes e depois da permutação. unsqueeze cria o lote sem alterar a ordem dos pixels. O valor 113 confirma uma localização específica; shape sozinho não detectaria uma reorganização errada. contiguous é usado após permute para tornar explícita a disposição desejada da memória.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 01 · EXERCÍCIOS AUTORAIS

1.3 Caderno de aplicação

Exercício A · Modificação controlada

Crie uma imagem com cinco bandas e converta HWC para NCHW. Escreva uma função que receba a ordem esperada das bandas e rejeite uma ordem diferente. Demonstre a diferença entre permute e reshape para reorganizar eixos.

Comentário de solução

permute troca a interpretação dos eixos mantendo os valores associados às posições; reshape reorganiza a forma conforme a ordem de armazenamento e não executa a mesma troca. No exemplo, o pixel da banda 2 em linha 4 e coluna 5 vale 113. A forma correta é [1,3,8,8]. A validação das bandas deve usar nomes ou códigos, não apenas o total de canais.

Exercício B · Aplicação ampliada

Adicione transformada afim e EPSG a um dicionário de metadados. Ao recortar uma janela, atualize a origem espacial correspondente; não copie a transformada do mosaico inteiro sem deslocamento. Teste a posição de um pixel de canto antes e depois do recorte.

Critérios específicos de revisão

Confirme formato, valor de referência e identificação das bandas. Número de canais correto não demonstra ordem correta.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 02 · TEORIA E MÉTODO

2. Normalização por banda e máscara de nodata

2.1 Fundamentos

A normalização deve usar estatísticas do treino e excluir pixels inválidos. Substituir nodata por zero antes de calcular média introduz um valor artificial que pode dominar uma banda. Algumas imagens têm máscaras diferentes por canal; o critério para pixel utilizável precisa ser declarado. A normalização não deve transformar um marcador de ausência em sinal temático.

Padronizar por média e desvio é uma opção, não uma regra universal. Reflectâncias já escaladas podem admitir outra transformação; bandas com distribuição assimétrica podem requerer clipping ou transformação específica. Os parâmetros precisam permanecer iguais na validação e na inferência. Calcular estatísticas de cada cena separadamente altera a referência do modelo e pode remover diferenças reais entre cenas.

O laboratório normaliza duas bandas e exclui um pixel por máscara comum. O desvio usa unbiased=False para descrever a distribuição dos pixels disponíveis, e clamp_min evita divisão por zero. Depois da transformação, nodata recebe zero somente como representação computacional acompanhada da máscara. Métricas e perda ainda devem ignorar esses pixels.

Hipóteses e aprofundamento técnico

Para acumular estatísticas em dados extensos, mantenha quantidade válida, média e segundo momento por banda com uma atualização numericamente estável. Somar x^2 e subtrair $média^2$ pode sofrer cancelamento em magnitudes altas. Máscaras por banda devem ter política própria quando algumas bandas faltam. Um pixel com uma banda inválida pode ser descartado integralmente ou tratado com um canal de validade, conforme o modelo. A escolha modifica o conjunto usado no treino. Ao persistir parâmetros, registre unidade original e escala anterior, para que a inferência não aplique divisão ou fator de reflectância duas vezes.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Estatísticas	Somente treino válido	Evita vazamento e nodata
Desvio	Populacional no conjunto de pixels	Declare a convenção
Banda constante	clamp mínimo	Diagnosticar ausência de variação
Pixel inválido	Zero mais máscara	Excluir de perda e métrica

Registre máscara, estatísticas por canal e conjunto usado no ajuste. Nunca recalcule parâmetros usando o teste.

CAPÍTULO 02 · PROGRAMAÇÃO

2.2 Laboratório executável

Arquivo incorporado: capitulo_02.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
x=torch.arange(32,dtype=torch.float32).reshape(1,2,4,4)
valido=torch.ones((1,1,4,4),dtype=torch.bool)
valido[:, :, 0, 0]=False
z=torch.zeros_like(x); medias=[]; desvios=[]
for c in range(2):
    v=x[:,c][valido[:,0]]
    media=v.mean(); desvio=v.std(unbiased=False).clamp_min(1e-6)
    z[:,c]=(x[:,c]-media)/desvio
    medias.append(float(media)); desvios.append(float(desvio))
z=torch.where(valido.expand_as(z),z,torch.zeros_like(z))
assert abs(float(z[:,0][valido[:,0]].mean()))<1e-6
assert z[0,0,0,0]==0
RESULTADO={'medias':medias,'desvios':desvios,'validos':int(valido.sum())}
print(RESULTADO)
```

Resultado de referência calculado

```
{"desvios": [4.320493698120117, 4.320493698120117], "medias": [8.0, 24.0], "validos": 15}
```

Leitura do código

A seleção `v=x[:,c][valido[:,0]]` remove nodata antes de estimar média e desvio. O broadcast de `valido` para todos os canais aplica a mesma máscara no exemplo. `torch.where` define a representação final dos pixels inválidos, e o teste da média normalizada usa somente os válidos. Não use a média da matriz inteira para conferir esse contrato.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

2.3 Caderno de aplicação

Exercício A · Modificação controlada

Troque o nodata por -9999 e calcule a média com e sem máscara. Crie uma banda constante e verifique se a normalização permanece finita. Explique por que a banda constante deve ser registrada como um problema de informação, mesmo quando clamp evita a exceção.

Comentário de solução

As médias válidas são 8 e 24, porque o primeiro pixel de cada banda é excluído. Incluir -9999 distorce violentamente a estatística. A banda constante normalizada vira zero; isso é computacionalmente estável, porém não acrescenta variação preditiva. O clamp protege o cálculo e não valida a qualidade da banda.

Exercício B · Aplicação ampliada

Implemente uma rotina que acumule contagem, soma e soma de quadrados de janelas do treino sem carregar o mosaico inteiro. Confronte os parâmetros com o cálculo integral e trate cancelamento numérico em dados de grande magnitude.

Critérios específicos de revisão

Registre máscara, estatísticas por canal e conjunto usado no ajuste. Nunca recalcule parâmetros usando o teste.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 03 · TEORIA E MÉTODO

3. Convolução, padding e campo receptivo

3.1 Fundamentos

Conv2d aplica filtros aprendidos em vizinhanças e compartilha os mesmos pesos ao longo da imagem. A saída espacial depende de kernel, stride, padding e dilation. Para uma dimensão, a fórmula é $\text{floor}((\text{entrada} + 2 * \text{padding} - \text{dilation} * (\text{kernel} - 1) - 1) / \text{stride} + 1)$. O número de canais de entrada precisa coincidir com o formato do tensor.

Padding preserva dimensões em certas configurações, mas acrescenta valores artificiais nas bordas. Um filtro perto do limite observa menos contexto real que no interior. Em mosaicos, recortar sem margem e inferir cada bloco pode produzir costuras. O campo receptivo cresce ao empilhar camadas; resolução da saída e contexto físico devem ser analisados juntos.

O exemplo fixa um kernel de média 3×3 para inspecionar a operação sem depender de aprendizado. A imagem contém um impulso central e a convolução distribui sua contribuição pela vizinhança. O teste de soma funciona porque o impulso está longe da borda. Um impulso no canto sofre efeito do padding, revelando que conservação não vale automaticamente para qualquer posição.

Hipóteses e aprofundamento técnico

O número de parâmetros de uma convolução padrão é $C_{out} * C_{in} * k_h * k_w$ mais C_{out} se houver bias. Isso não cresce com H e W, ao contrário de uma camada linear aplicada ao flatten da imagem. O campo receptivo efetivo de camadas encadeadas depende também dos strides anteriores: cada novo kernel amplia o contexto em passos maiores após downsampling. Padding simétrico mantém forma em certos casos, mas cria condições de contorno. Ao inferir recortes, inclua margem de contexto ou compare apenas a região central correspondente ao campo observado em treino.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Kernel 3	Vizinhança 3 × 3	Contexto local
Padding 1	Mantém dimensão com stride 1	Borda artificial
Stride 2	Reduz resolução	Pode perder detalhe
Dilation	Amplia espaçamento	Aumenta campo receptivo

Calcule forma e contexto antes de executar. Revise bordas e escala física, especialmente em inferência por blocos.

CAPÍTULO 03 · PROGRAMAÇÃO

3.2 Laboratório executável

Arquivo incorporado: capitulo_03.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
from torch import nn
x=torch.zeros((1,1,7,7)); x[0,0,3,3]=9
conv=nn.Conv2d(1,1,kernel_size=3,padding=1,bias=False)
with torch.no_grad():
    conv.weight.fill_(1/9)
    y=conv(x)
assert y.shape==x.shape and abs(float(y.sum())-9)<1e-6
assert float(y[0,0,3,3])==1
RESULTADO={'saida':list(y.shape),'soma':float(y.sum()),
'pixels_nao_zero':int((y!=0).sum())}
print(RESULTADO)
```

Resultado de referência calculado

```
{"pixels_nao_zero": 9, "saida": [1, 1, 7, 7], "soma": 9.0}
```

Leitura do código

O kernel recebe pesos fixos 1/9 dentro de no_grad, portanto a operação testa geometria da convolução e não ajuste. O impulso central distribui valor um em nove posições. bias=False impede uma contribuição constante à saída. A asserção de soma é válida para o impulso interior e serve de contraste com o experimento de canto.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

3.3 Caderno de aplicação

Exercício A · Modificação controlada

Mova o impulso para o canto e calcule a soma da saída. Troque padding por zero e confira a forma usando a fórmula. Empilhe duas convoluções 3×3 e determine o campo receptivo efetivo no interior.

Comentário de solução

No canto, apenas quatro posições da saída recebem o impulso, totalizando 4 em vez de 9 para o kernel de média usado. Sem padding, a saída de entrada 7×7 e kernel 3×3 é 5×5. Duas camadas 3×3 com stride 1 têm campo receptivo 5×5, pois a segunda observa vizinhanças de saídas que já dependem de três pixels por dimensão.

Exercício B · Aplicação ampliada

Converta campo receptivo para metros usando resolução de 10 e 30 metros por pixel. A mesma arquitetura observa extensões físicas diferentes; ajustar sensores sem considerar escala pode mudar a natureza do padrão aprendido.

Critérios específicos de revisão

Calcule forma e contexto antes de executar. Revise bordas e escala física, especialmente em inferência por blocos.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 04 · TEORIA E MÉTODO

4. Pooling e classificação de patches

4.1 Fundamentos

Uma classificação de patch produz um rótulo por recorte. Isso difere de segmentação, que produz um rótulo por pixel. AdaptiveAvgPool2d reduz mapas de atributos a uma dimensão fixa, permitindo um classificador com entrada espacial variável dentro de condições de treino compatíveis. A média espacial resume ocorrência de padrões, mas perde sua localização detalhada.

Uma arquitetura pequena pode combinar convolução, ReLU, média global e camada linear. A camada final retorna logits, não probabilidades. CrossEntropyLoss aplica a transformação apropriada internamente; aplicar softmax antes dessa perda pode prejudicar o cálculo. Para apresentar probabilidades, use softmax apenas na avaliação.

O laboratório define uma CNN de três bandas e duas classes e confere dimensões e normalização das probabilidades. Pesos aleatórios não constituem um classificador útil: a saída inicial apenas testa a arquitetura. A escolha de rótulo do patch precisa ser consistente; maioria de pixels, presença mínima e classe central são contratos diferentes e geram alvos distintos.

Hipóteses e aprofundamento técnico

Uma média global torna a rede menos dependente da localização exata dos padrões, mas também pode remover informação necessária para contar ou localizar objetos. Classificar um patch como urbano não produz um mapa de construção individual. O tamanho físico do patch é $H \times \text{resolução}$ por $W \times \text{resolução}$; mudar dimensão ou sensor altera esse suporte. O total de classes da última camada precisa coincidir com a codificação do alvo. CrossEntropyLoss recebe índices inteiros por exemplo, enquanto probabilidades de apresentação vêm de softmax sobre a dimensão de classes, nunca sobre altura ou lote.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Classificação	Um rótulo por patch	Não localiza objetos
Pooling global	Média de mapas	Reduz informação de posição
Logits	Escore sem normalização	Entrada de CrossEntropyLoss
Softmax	Probabilidades por classe	Aplicar na apresentação

Conserve logits antes da perda, confira total de parâmetros e declare o significado do rótulo do patch.

CAPÍTULO 04 · PROGRAMAÇÃO

4.2 Laboratório executável

Arquivo incorporado: capitulo_04.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
from torch import nn
torch.manual_seed(24)
modelo=nn.Sequential(nn.Conv2d(3,8,3,padding=1),nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),nn.Flatten(),nn.Linear(8,2))
x=torch.rand((4,3,16,16))
logits=modelo(x); p=logits.softmax(dim=1)
assert logits.shape==(4,2)
assert torch.allclose(p.sum(dim=1),torch.ones(4),atol=1e-6)
RESULTADO={'logits':list(logits.shape),
    'parametros':sum(v.numel() for v in modelo.parameters()),
    'classes_iniciais':p.argmax(dim=1).tolist()}
print(RESULTADO)
```

Resultado de referência calculado

```
{"classes_iniciais": [0, 0, 0, 0], "logits": [4, 2], "parametros": 242}
```

Leitura do código

A primeira convolução produz oito mapas, ReLU aplica não linearidade e a média global deixa oito números por imagem. Flatten remove apenas dimensões unitárias depois do pooling. A camada linear retorna dois logits. `p.sum(dim=1)` confere normalização entre classes de cada imagem; somar em outra dimensão verificaria uma propriedade diferente.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 04 · EXERCÍCIOS AUTORAIS

4.3 Caderno de aplicação

Exercício A · Modificação controlada

Troque o patch por 32×32 e confirme a mesma dimensão dos logits. Substitua a média global por flatten direto e explique por que a dimensão da camada linear passa a depender do tamanho espacial. Calcule o número de parâmetros da rede original.

Comentário de solução

A rede original contém 224 parâmetros na convolução e 18 na camada linear, totalizando 242. AdaptiveAvgPool2d(1) produz oito valores por imagem, independentemente de 16×16 ou 32×32 . Flatten sem pooling produz $8 \times \text{altura} \times \text{largura}$ valores e exige uma camada linear compatível. Aceitar tamanhos diferentes matematicamente não demonstra que o modelo generaliza a outra escala física.

Exercício B · Aplicação ampliada

Defina três políticas de rótulo para um patch com 10% de superfície construída: presença, maioria e pixel central. Crie máscaras que mostrem quando os rótulos divergem e escolha uma política de acordo com a aplicação.

Critérios específicos de revisão

Conserve logits antes da perda, confira total de parâmetros e declare o significado do rótulo do patch.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 05 · TEORIA E MÉTODO

5. Encoder-decoder e conexão de resolução

5.1 Fundamentos

Uma arquitetura encoder-decoder combina contexto em resolução reduzida com reconstrução da saída por pixel. O encoder transforma a imagem em mapas de atributos; pooling reduz a dimensão espacial e amplia o contexto efetivo. O decoder recupera a resolução e produz logits. Uma conexão skip transporta informação em alta resolução para reduzir a perda de detalhe de bordas.

Upsampling não recupera magicamente informação descartada. Interpolação bilinear estima valores intermediários dos mapas de atributos; não deve ser usada para interpolar classes discretas. Concatenar skip e decoder aumenta canais e exige uma convolução com dimensão correspondente. Tamanhos ímpares podem produzir incompatibilidade após pooling; interpolar para o tamanho explícito da skip resolve a forma, sem eliminar a necessidade de verificar alinhamento.

O laboratório implementa uma rede reduzida inspirada no mecanismo de encoder-decoder, sem alegar reproduzir uma U-Net completa. A entrada possui altura ímpar para exercitar ajuste de forma. A perda binária propaga gradiente por todos os caminhos, inclusive o skip. Esse teste verifica arquitetura e diferenciação, mas um modelo de segmentação real ainda demanda dados, treino e avaliação por cena.

Hipóteses e aprofundamento técnico

Um caminho de resolução reduzida amplia contexto, e o skip oferece atributos locais ao decoder. A concatenação ocorre em canais e conserva altura e largura compatíveis. A perda precisa propagar pelo encoder; um detach indevido no skip alteraria o treinamento sem necessariamente mudar a forma. Em redes maiores, múltiplos níveis exigem contabilizar memória de ativações, especialmente ao conservar skips. Interpolação com align_corners=False define uma convenção própria de posicionamento dos valores; ela deve ser mantida entre treino e inferência. Teste entradas com dimensões ímpares para expor supostos ocultos de divisibilidade.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Encoder	3 → 4 canais	Mapas em resolução original
Fundo	4 → 8 após pooling	Contexto reduzido
Skip + up	4 + 8 = 12 canais	Conserva detalhe e contexto
Decoder	12 → 1 logit	Saída binária por pixel

Confira canais concatenados, tamanho da saída, gradiente em encoder e decoder e alinhamento antes de treinar.

CAPÍTULO 05 · PROGRAMAÇÃO

5.2 Laboratório executável

Arquivo incorporado: capitulo_05.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
from torch import nn
from torch.nn import functional as F
class Rede(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc=nn.Conv2d(3,4,3,padding=1)
        self.fundo=nn.Conv2d(4,8,3,padding=1)
        self.dec=nn.Conv2d(12,1,3,padding=1)
    def forward(self,x):
        skip=F.relu(self.enc(x))
        fundo=F.relu(self.fundo(F.max_pool2d(skip,2)))
        up=F.interpolate(fundo,size=skip.shape[-2:],
                        mode='bilinear',align_corners=False)
        return self.dec(torch.cat([skip,up],dim=1))
torch.manual_seed(33)
m=Rede(); x=torch.rand(2,3,15,17); y=torch.zeros(2,1,15,17)
logits=m(x); loss=nn.BCEWithLogitsLoss()(logits,y); loss.backward()
assert logits.shape==y.shape and torch.isfinite(m.enc.weight.grad).all()
RESULTADO={'saida':list(logits.shape),
            'parametros':sum(v.numel() for v in m.parameters()),
            'perda':round(float(loss.detach()),4)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"parametros": 517, "perda": 0.6965, "saida": [2, 1, 15, 17]}
```

Leitura do código

O encoder gera quatro canais, pooling reduz resolução e a camada de fundo gera oito. O upsample retorna ao tamanho exato da skip e cat cria doze canais. A camada final produz um logit por pixel. A asserção no gradiente do encoder confirma que a perda alcançou o início da rede e permaneceu finita.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

5.3 Caderno de aplicação

Exercício A · Modificação controlada

Remova a conexão skip e adapte self.dec para oito canais. Compare número de parâmetros e forma final. Depois substitua interpolação por ConvTranspose2d e teste entradas pares e ímpares, registrando onde aparecem diferenças de tamanho.

Comentário de solução

A conexão skip acrescenta quatro canais à convolução final. Removê-la reduz parâmetros dessa etapa, mas também elimina o caminho direto de atributos em alta resolução. Uma convolução transposta exige kernel, stride, padding e output_padding compatíveis com o tamanho desejado; dimensões ímpares não devem ser corrigidas por recorte arbitrário sem revisar alinhamento. No laboratório, a interpolação usa o tamanho explícito do skip.

Exercício B · Aplicação ampliada

Treine a rede em máscaras sintéticas com bordas finas e compare IoU com uma convolução 1×1. Separe imagens inteiras antes de extrair patches e use a mesma perda mascarada. Registre se o ganho vem de contexto ou apenas de maior capacidade de ajuste.

Critérios específicos de revisão

Confira canais concatenados, tamanho da saída, gradiente em encoder e decoder e alinhamento antes de treinar.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 06 · TEORIA E MÉTODO

6. Treinamento, gradientes e otimizador

6.1 Fundamentos

Treinar uma rede envolve calcular saída, perda, gradientes e atualizar pesos. `zero_grad` elimina gradientes acumulados da iteração anterior; `backward` calcula derivadas pela cadeia computacional; `step` aplica a regra do otimizador. Esquecer `zero_grad` modifica o algoritmo sem necessariamente provocar erro de execução.

Adam combina momentos dos gradientes e uma escala adaptativa por parâmetro. A taxa de aprendizado continua decisiva: muito alta pode causar oscilação; muito baixa pode tornar o treino lento. Redução da perda de treino confirma capacidade de ajuste naquele conjunto, não generalização. Separe um conjunto de validação e escolha época ou configuração sem consultar o teste final.

O laboratório constrói patches cujo brilho de duas bandas diferencia classes. Uma rede pequena aprende esse sinal em CPU. A comparação entre perda inicial e final é um teste de sanidade para o fluxo completo. Não use esse conjunto fácil para reivindicar desempenho em imagens reais, onde textura, nuvens, mistura espectral e mudança de domínio tornam o problema mais difícil.

Hipóteses e aprofundamento técnico

Gradientes acumulados podem ser usados deliberadamente para simular lotes maiores, mas a perda e frequência de `step` devem ser planejadas para conservar escala. Esse procedimento difere de esquecer `zero_grad` entre épocas. Em checkpoint de retomada, estado do otimizador também importa para Adam, pois momentos influenciam passos seguintes. Para inferência, apenas pesos e buffers podem bastar. Compare perda média por observação quando lotes têm tamanhos diferentes. Um treinamento que reduz perda em todas as épocas ainda pode sobreajustar; escolha checkpoint pelo critério de validação definido previamente.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
<code>zero_grad</code>	Limpar acumulação	Executar antes de <code>backward</code>
<code>backward</code>	Derivadas da perda	Exige grafo de treino
<code>step</code>	Atualização de parâmetros	Usa gradientes correntes
Validação	Dados não ajustados	Não substituída pela perda de treino

Controle `seed` e `threads`, confirme gradientes finitos e acompanhe treino e validação separadamente.

CAPÍTULO 06 · PROGRAMAÇÃO

6.2 Laboratório executável

Arquivo incorporado: capitulo_06.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
from torch import nn
torch.set_num_threads(1); torch.manual_seed(25)
y=torch.arange(16)%2
x=torch.rand(16,2,8,8)*.1+y[:,None,None,None].float()*.8
m=nn.Sequential(nn.Conv2d(2,4,3,padding=1),nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),nn.Flatten(),nn.Linear(4,2))
opt=torch.optim.Adam(m.parameters(),lr=.03)
perda=nn.CrossEntropyLoss(); inicial=float(perda(m(x),y).detach())
m.train()
for _ in range(35):
    opt.zero_grad(); loss=perda(m(x),y)
    loss.backward(); opt.step()
final=float(perda(m(x),y).detach())
assert final<inicial and torch.isfinite(loss)
RESULTADO={'perda_inicial':round(inicial,4),'perda_final':round(final,4)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"perda_final": 0.0166, "perda_inicial": 0.7219}
```

Leitura do código

A classe `y` determina o brilho das duas bandas com ruído pequeno. A rede é ajustada no mesmo lote para um teste de overfit controlado, útil para verificar gradiente e atualização. Cada iteração limpa gradientes antes de `backward`. O resultado inicial e final é extraído com `detach` para não registrar um novo caminho de otimização na leitura.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 06 · EXERCÍCIOS AUTORAIS

6.3 Caderno de aplicação

Exercício A · Modificação controlada

Remova `zero_grad` e compare curva de perda e norma dos gradientes. Depois reduza a taxa para 0,0001 e mantenha 35 épocas. Registre se o teste de diminuição continua satisfeito e explique por que a intensidade da melhoria muda.

Comentário de solução

Sem `zero_grad`, as contribuições de épocas anteriores acumulam e o experimento deixa de executar o mesmo algoritmo. Com taxa menor, os pesos se movem menos a cada passo; uma perda final ligeiramente menor pode satisfazer a asserção sem produzir classificação útil. A verificação de sanidade confirma operação do treinamento, mas precisa ser complementada por validação independente.

Exercício B · Aplicação ampliada

Separe quatro patches como validação antes do treino, registre loss em ambos os conjuntos e salve a melhor época por validação. Altere o brilho na validação para examinar mudança de domínio sem reajustar normalização com esses dados.

Critérios específicos de revisão

Controle seed e threads, confirme gradientes finitos e acompanhe treino e validação separadamente.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 07 · TEORIA E MÉTODO

7. Segmentação binária e perdas estáveis

7.1 Fundamentos

Na segmentação binária, o modelo retorna um logit por pixel. O alvo normalmente é um tensor float com valores zero e um e a mesma forma espacial. BCEWithLogitsLoss combina sigmoid e entropia cruzada binária de forma numericamente estável. Aplicar sigmoid antes dessa perda transforma incorretamente a entrada esperada.

Pixels nodata não devem contribuir à perda. Com `reduction="none"`, a perda é calculada por elemento e pode ser ponderada por máscara. Dividir pela quantidade de pixels válidos, em vez do número total, mantém a escala coerente quando muda a cobertura. Um lote sem pixel válido exige uma política de rejeição ou skip: clamp no denominador sozinho não cria informação de treino.

O laboratório utiliza uma convolução 1×1 , adequada para testar classificação por espectro sem contexto espacial. Uma rede real pode adicionar convoluções e caminhos de resolução, mas o contrato da saída permanece. O gradiente finito confirma que logits, alvos e máscara participam do cálculo correto. A resolução e alinhamento dos rótulos precisam ser verificados antes de qualquer otimização.

Hipóteses e aprofundamento técnico

A perda binária por logit pode ser expressa de forma estável por $\max(z, 0) - z * y + \log(1 + \exp(-\text{abs}(z)))$. Essa forma evita calcular diretamente $\log(\text{sigmoid}(z))$ em extremos. Ponderar pela máscara e dividir por sua soma mantém média sobre observações válidas. `pos_weight` modifica contribuição positiva, útil para desbalanceamento, mas não elimina necessidade de avaliar probabilidade e limiar. Em segmentação multiclasse, CrossEntropyLoss recebe logits $N \times K \times H \times W$ e alvos $N \times H \times W$ com índices de classe; esse contrato é diferente do binário com um canal.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Logit	$N \times 1 \times H \times W$	Sem sigmoid antes da perda
Alvo	Float zero ou um	Mesmo alinhamento espacial
Máscara	Peso por pixel	Nodata não contribui
Denominador	Pixels válidos	Rejeitar lote totalmente inválido

Confira forma, dtype e alinhamento. Exclua nodata tanto da perda quanto das métricas.

CAPÍTULO 07 · PROGRAMAÇÃO

7.2 Laboratório executável

Arquivo incorporado: capitulo_07.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
from torch import nn
torch.manual_seed(26)
x=torch.rand(2,3,8,8)
y=(x[:,0:1]>.5).float()
valido=torch.ones_like(y); valido[:, :, 0, :]=0
m=nn.Conv2d(3,1,1); logits=m(x)
por_pixel=nn.BCEWithLogitsLoss(reduction='none')(logits,y)
assert valido.sum()>0
loss=(por_pixel*valido).sum()/valido.sum()
loss.backward()
assert torch.isfinite(m.weight.grad).all()
RESULTADO={'pixels_validos':int(valido.sum()),
'loss':round(float(loss.detach()),4),'saida':list(logits.shape)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"loss": 0.7295, "pixels_validos": 112, "saida": [2, 1, 8, 8]}
```

Leitura do código

`x[:,0:1]` conserva dimensão de canal e produz alvo com mesma forma da saída. A primeira linha é inválida em ambos os patches, resultando em 112 pixels avaliados. `reduction="none"` deixa disponível a perda por pixel antes de multiplicar a máscara. `backward` usa a média válida e a asserção confere gradiente dos pesos.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

7.3 Caderno de aplicação

Exercício A · Modificação controlada

Marque todo o primeiro patch como inválido e confira o denominador. Depois marque ambos como inválidos e faça a rotina lançar ValueError. Insira um deslocamento de um pixel no alvo e explique como ele altera a fronteira aprendida.

Comentário de solução

Com o primeiro patch inválido, a perda considera apenas os 56 pixels válidos do segundo. Com tudo inválido, não existe evidência para treinar; lançar erro ou pular explicitamente o lote é melhor que retornar uma perda enganosa igual a zero. Um alvo deslocado ensina relações espaciais incorretas, mesmo quando sua distribuição de classes continua idêntica.

Exercício B · Aplicação ampliada

Compare a convolução 1×1 com uma 3×3 em um alvo gerado por vizinhança. A primeira só observa espectro local e não representa a mesma hipótese. Registre pixel size, alinhamento afim e origem de cada máscara de treinamento.

Critérios específicos de revisão

Confira forma, dtype e alinhamento. Exclua nodata tanto da perda quanto das métricas.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 08 · TEORIA E MÉTODO

8. Augmentação geométrica conjunta

8.1 Fundamentos

Augmentação aplica transformações para ampliar a variedade de exemplos mantendo a relação entre imagem e alvo. Em segmentação, flip ou rotação precisam ocorrer em ambos, com a mesma decisão aleatória. Transformar somente a imagem destrói o alinhamento e gera supervisão inconsistente. Máscaras categóricas não devem receber interpolação bilinear.

As transformações precisam respeitar a semântica. Rotação pode ser válida para padrões sem orientação privilegiada e inadequada quando azimuth, iluminação ou direção de relevo fazem parte da informação. Alterar brilho de bandas individualmente pode modificar índices espectrais e criar amostras fisicamente implausíveis. Defina uma política que represente variação plausível, não apenas disponibilidade de funções.

O laboratório aplica flip horizontal e rotação de 90 graus à imagem e à máscara. O teste preserva a regra construída entre primeira banda e alvo. Essas operações são permutações exatas de pixels, portanto não introduzem interpolação. Em um projeto georreferenciado, a augmentação serve ao treino de patches; não copie a transformada espacial original como se a imagem aumentada continuasse no mesmo posicionamento de mapa.

Hipóteses e aprofundamento técnico

Uma transformação geométrica de treino deve preservar relação entre todas as camadas: bandas, rótulos e validade. Para operações que interpolam imagem, o interpolador de imagem pode diferir do interpolador de classe, mas a transformação espacial deve ser a mesma. Augmentação não aumenta número de cenas independentes; várias variantes de uma imagem continuam compartilhando sua origem e precisam ficar na mesma divisão. Políticas de rotação também podem exigir transformar canais de orientação ou vetores, pois rotacionar apenas a imagem altera sua relação com atributos direcionais.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Flip	Imagem e alvo juntos	Preserva correspondência
Rotação 90°	Permutação exata	Não interpola categorias
Resize de máscara	Nearest neighbor	Evita classes fracionárias
Perturbação espectral	Política por sensor	Não inventar reflectância

Mantenha decisões aleatórias compartilhadas e preserve categorias inteiras. Declare limites físicos da política de augmentação.

CAPÍTULO 08 · PROGRAMAÇÃO

8.2 Laboratório executável

Arquivo incorporado: capitulo_08.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
imagem=torch.arange(16, dtype=torch.float32).reshape(1,4,4)
mascara=(imagem[0]>=8).long()
img_flip=torch.flip(imagem, dims=[2])
mask_flip=torch.flip(mascara, dims=[1])
img_rot=torch.rot90(img_flip, k=1, dims=[1,2])
mask_rot=torch.rot90(mask_flip, k=1, dims=[0,1])
assert torch.equal((img_rot[0]>=8).long(), mask_rot)
assert int(mask_rot.sum())==8
RESULTADO={'positivos':int(mask_rot.sum()),
            'alinhamento_preservado':True, 'canto':float(img_rot[0,0,0])}
print(RESULTADO)
```

Resultado de referência calculado

```
{"alinhamento_preservado": true, "canto": 0.0, "positivos": 8}
```

Leitura do código

A imagem tem um canal e a máscara não tem dimensão de canal. Por isso dims do flip e da rotação são diferentes, embora a transformação espacial seja equivalente. A comparação após as duas operações testa uma regra de alinhamento. Contar positivos é complementar e detecta perda de categoria, mas sozinho não identifica pixels trocados.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

8.3 Caderno de aplicação

Exercício A · Modificação controlada

Aplique o flip somente à imagem e construa uma máscara com positivo apenas em uma coluna, para que o erro fique visível. Compare número total de positivos e correspondência pixel a pixel; mostre que conservar contagem não garante alinhamento.

Comentário de solução

A contagem de classe permanece igual sob flip, mesmo quando imagem e máscara ficam desalinhadas. Por isso um teste de histograma não substitui um teste de posição. Com uma máscara assimétrica, a comparação pixel a pixel detecta o problema. No exemplo correto, oito positivos permanecem associados aos mesmos valores transformados da primeira banda.

Exercício B · Aplicação ampliada

Implemente uma função que sorteie a transformação uma vez e a aplique a imagem, máscara de classe e máscara de validade. Escreva testes para identidade, flip e rotação. Restrinja transformações quando houver uma variável de orientação que não possa ser mantida.

Critérios específicos de revisão

Mantenha decisões aleatórias compartilhadas e preserve categorias inteiras. Declare limites físicos da política de augmentação.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 09 · TEORIA E MÉTODO

9. Métricas de segmentação e classe rara

9.1 Fundamentos

IoU é a razão entre interseção e união dos conjuntos predito e observado. Dice usa duas vezes a interseção dividida pela soma das quantidades positivas. Ambas descrevem sobreposição, mas não são idênticas. Acurácia pixel a pixel pode ser alta quando a classe construída ocupa uma pequena parte da cena e é ignorada pelo modelo.

O cálculo precisa excluir nodata e definir comportamento quando não há positivos nem predições. Algumas convenções atribuem um à correspondência vazia; outras omitem o caso. Sem declarar a convenção, médias entre cenas podem ser incomparáveis. Métricas agregadas por pixel dão maior peso a cenas grandes; média por cena atribui o mesmo peso a cada território.

O laboratório constrói uma pequena máscara, calcula TP, FP e FN apenas onde válido e deriva IoU e Dice. A asserção verifica valores conhecidos, tornando o exercício conferível à mão. Em produção, reporte também quantidade de pixels válidos, resolução, área da classe e desempenho por cena. Uma diferença de um pixel tem significado físico diferente em sensores de 1 e 30 metros.

Hipóteses e aprofundamento técnico

IoU e Dice mantêm relação $Dice=2*IoU/(1+IoU)$ quando derivadas das mesmas contagens binárias. A identidade permite um teste adicional da implementação. Limiar altera TP, FP e FN e deve ser selecionado fora do teste. Máscaras com tolerância de borda avaliam uma pergunta diferente da segmentação exata. Se houver erro posicional do rótulo, faça uma análise separada e documente resolução e tolerância, sem inflar silenciosamente a métrica. Também registre área física de erro, usando resolução e referência adequada, para que a pontuação tenha significado territorial.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
IoU	$TP / (TP+FP+FN)$	Sobreposição por união
Dice	$2TP / (2TP+FP+FN)$	Peso duplo da interseção
Acurácia	$(TP+TN)/total$	Sensível à classe majoritária
Nodata	Excluir de todas as contagens	Não tratar como negativo

Declare nodata, limiar e regra do caso vazio. Reporte contagens por cena para interpretar a métrica.

CAPÍTULO 09 · PROGRAMAÇÃO

9.2 Laboratório executável

Arquivo incorporado: capitulo_09.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
y=torch.tensor([[1,1,0],[0,1,0],[0,0,0]],dtype=torch.bool)
p=torch.tensor([[1,0,1],[0,1,0],[0,0,1]],dtype=torch.bool)
v=torch.ones_like(y); v[2,2]=False
tp=int((y&p&v).sum()); fp=int((~y&p&v).sum())
fn=int((y&~p&v).sum())
iou=tp/(tp+fp+fn); dice=2*tp/(2*tp+fp+fn)
assert tp==2 and fp==1 and fn==1
assert iou==.5 and abs(dice-2/3)<1e-8
RESULTADO={'TP':tp,'FP':fp,'FN':fn,'IoU':iou,'Dice':round(dice,4)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"Dice": 0.6667, "FN": 1, "FP": 1, "IoU": 0.5, "TP": 2}
```

Leitura do código

Operadores booleanos combinam alvo, predição e validade em cada contagem. O falso positivo no canto inferior direito é excluído por v. O cálculo não usa softmax ou uma rede para isolar o teste da métrica. A relação entre Dice e IoU confirma internamente as mesmas contagens e ajuda a detectar denominadores inconsistentes.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

9.3 Caderno de aplicação

Exercício A · Modificação controlada

Inclua o pixel (2,2) na avaliação e recalcule métricas. Depois acrescente cem pixels negativos corretamente classificados e compare IoU com acurácia. Explique por que a acurácia aumenta sem melhoria na detecção da classe positiva.

Comentário de solução

Incluir o pixel antes inválido adiciona um falso positivo: IoU passa a $2/5=0,4$ e Dice a $4/7$. Acrescentar negativos corretos aumenta TN e acurácia, mas não altera TP, FP ou FN; IoU e Dice permanecem iguais. A avaliação da classe rara deve mostrar explicitamente detecção e falsas inclusões, e não apenas o desempenho dominado pelo fundo.

Exercício B · Aplicação ampliada

Calcule média de IoU por cena e IoU global a partir da soma de contagens. Use cenas com tamanhos diferentes para mostrar que as duas agregações têm pesos distintos. Defina e teste a convenção do caso vazio.

Critérios específicos de revisão

Declare nodata, limiar e regra do caso vazio. Reporte contagens por cena para interpretar a métrica.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 10 · TEORIA E MÉTODO

10. Validação por cena e modos train e eval

10.1 Fundamentos

Patches extraídos de uma mesma cena compartilham iluminação, sensor, processamento e contexto. Dividi-los aleatoriamente pode colocar recortes quase idênticos em treino e teste, especialmente com janelas sobrepostas. Separe cenas, regiões ou datas antes da extração dos patches conforme a pergunta de generalização.

`model.eval` altera o comportamento de camadas como Dropout e BatchNorm; `torch.no_grad` ou `inference_mode` desativa registro de gradientes para inferência. São controles diferentes e complementares. `eval` não impede a criação de grafo sozinho, e `no_grad` não muda automaticamente o estado de Dropout. Métricas consistentes exigem os dois cuidados quando essas camadas estão presentes.

O laboratório retém uma cena e usa Dropout para demonstrar repetibilidade da inferência em modo avaliação. O teste com outputs iguais verifica esse comportamento, não a qualidade de uma arquitetura. Estatísticas de BatchNorm aprendidas no treino precisam permanecer preservadas; recalculá-las no teste também pode alterar o experimento. Use manifestos de divisão para impedir que uma cena reapareça por nomes diferentes.

Hipóteses e aprofundamento técnico

Uma divisão por cena precisa considerar sobreposição de aquisições e produtos derivados da mesma fonte. Dois nomes de arquivo diferentes podem conter áreas praticamente idênticas. Manifestos devem incluir origem, data, footprint e relação com produtos derivados. Seeds controlam parte da aleatoriedade, mas nem toda operação em GPU é determinística em qualquer versão; configure e documente quando a reprodução exigir isso. Em avaliação, mantenha parâmetros de BatchNorm e normalização aprendidos no treino. Atualizar estatísticas com o teste altera a pergunta e precisa ser tratado como adaptação específica, com avaliação distinta.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Por patch	Pode compartilhar cena	Risco de vazamento
Por cena	Retém origem inteira	Teste de domínio
eval	Desativa Dropout de treino	Não desativa grafo sozinho
inference_mode	Sem grafo de gradiente	Não troca modo da rede

Separe a origem antes dos patches e combine eval com `inference_mode`. Inspecione nomes, datas e duplicatas da divisão.

CAPÍTULO 10 · PROGRAMAÇÃO

10.2 Laboratório executável

Arquivo incorporado: capitulo_10.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
from torch import nn
cenas=['A','A','B','B','C','C']
treino=[i for i,c in enumerate(cenas) if c!='C']
teste=[i for i,c in enumerate(cenas) if c=='C']
assert set(cenas[i] for i in treino).isdisjoint(cenas[i] for i in teste)
torch.manual_seed(30)
m=nn.Sequential(nn.Dropout(.5),nn.Linear(4,2))
x=torch.ones(2,4); m.eval()
with torch.inference_mode():
    a=m(x); b=m(x)
assert torch.equal(a,b)
RESULTADO={'treino':treino,'teste':teste,
    'inferencias_iguais':True,'gradiente':a.requires_grad}
print(RESULTADO)
```

Resultado de referência calculado

```
{"gradiente": false, "inferencias_iguais": true, "teste": [4, 5], "treino": [0, 1, 2, 3]}
```

Leitura do código

A seleção de índices é feita pelo identificador de cena, não pelo valor do patch. `eval` desativa o Dropout de treino e `inference_mode` remove acompanhamento de gradientes. A comparação de outputs iguais usa exatamente a mesma entrada. Os pesos não foram treinados; o exercício demonstra modo de operação e independência da divisão, não qualidade de classificação.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 10 · EXERCÍCIOS AUTORAIS

10.3 Caderno de aplicação

Exercício A · Modificação controlada

Execute duas inferências com `m.train()` e `no_grad` e compare outputs. Depois use `m.eval()` sem `no_grad` e observe `requires_grad`. Redija um protocolo para separar imagens de datas distintas de uma mesma região.

Comentário de solução

Em `train`, Dropout sorteia máscaras mesmo dentro de `no_grad`; duas saídas podem divergir. Em `eval`, Dropout fica inativo, mas operações ainda podem registrar gradiente se os parâmetros o exigirem. No laboratório correto, as inferências são iguais e `requires_grad=False`. Separar por data testa transferência temporal; separar por região testa outra dimensão de generalização.

Exercício B · Aplicação ampliada

Inclua identificador de aquisição e região no manifesto de patches. Verifique duplicatas por hash e sobreposição de janelas entre divisões. Defina uma margem espacial quando recortes de regiões vizinhas compartilhareм contexto.

Critérios específicos de revisão

Separe a origem antes dos patches e combine `eval` com `inference_mode`. Inspecione nomes, datas e duplicatas da divisão.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 11 · TEORIA E MÉTODO

11. Inferência por janelas e reconstrução sem costuras

11.1 Fundamentos

Mosaicos extensos não cabem sempre na memória. A inferência por janelas processa blocos e reconstrói a saída. Janelas sobrepostas podem reduzir efeito de bordas se previsões forem combinadas por pesos adequados. Somar previsões sem dividir pela quantidade de contribuições aumenta valores nas áreas de sobreposição.

O acumulador deve manter soma de probabilidades e soma de pesos por pixel. Ao final, divide-se apenas onde há cobertura válida. Bordas da imagem precisam de uma política de padding ou janelas ajustadas, e todos os pixels pretendidos devem receber ao menos uma contribuição. Máscaras de nodata entram também nos pesos; uma região não observada não deve ganhar probabilidade por preenchimento implícito.

O laboratório reconstrói uma imagem 8×8 a partir de nove janelas 4×4 com passo dois. Usa identidade em vez de uma rede para isolar o mecanismo de mosaico: a saída deve recuperar exatamente a imagem. Depois, a operação de previsão pode ser substituída por sigmoid(modelo(patch)) sem mudar a lógica de acumulação. Transformada afim e extensão devem acompanhar a gravação geoespacial.

Hipóteses e aprofundamento técnico

Combinar probabilidades antes do limiar preserva informação contínua e permite uma decisão consistente no mosaico. Votar classes por janela é um método diferente e pode produzir fronteiras distintas. Pesos suaves reduzem contribuição de bordas, mas precisam cobrir o limite externo. Se os logits forem combinados em vez de probabilidades, a transformação final não será em geral igual à média de sigmoids; declare a escolha. Para rasters grandes, acumuladores em disco ou escrita por blocos exigem cuidado para não sobrescrever contribuições e para manter a mesma transformada espacial em todas as janelas.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Soma	Acumular probabilidades	Não aplicar limiar por janela
Peso	Quantidade ou janela ponderada	Normaliza sobreposição
Cobertura	Peso positivo	Detecta pixels esquecidos
Saída espacial	CRS e transformada	Conservar referência do mosaico

Confira cobertura completa e erro de reconstrução da identidade antes de atribuir costuras ao modelo.

CAPÍTULO 11 · PROGRAMAÇÃO

11.2 Laboratório executável

Arquivo incorporado: capitulo_11.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import torch
imagem=torch.arange(64, dtype=torch.float32).reshape(8,8)/63
soma=torch.zeros_like(imagem); peso=torch.zeros_like(imagem)
for r in range(0,5,2):
    for c in range(0,5,2):
        patch=imagem[r:r+4,c:c+4]
        previsao=patch.clone() # Identidade para testar o mosaico.
        soma[r:r+4,c:c+4]+=previsao
        peso[r:r+4,c:c+4]+=1
assert (peso>0).all()
saida=soma/peso
assert torch.allclose(saida,imagem,atol=1e-6)
RESULTADO={'erro_max':float((saida-imagem).abs().max()),
            'contribuicoes_max':int(peso.max()), 'cobertura':int((peso>0).sum())}
print(RESULTADO)
```

Resultado de referência calculado

```
{"cobertura": 64, "contribuicoes_max": 4, "erro_max": 0.0}
```

Leitura do código

As posições 0, 2 e 4 em cada eixo formam nove janelas. Peso registra entre uma e quatro contribuições por pixel e soma guarda os valores correspondentes. A divisão reconstrói identidade sem costuras. A rotina de inferência real deve substituir apenas previsao, mantendo a máscara e a lógica de acumulação testadas.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 11 · EXERCÍCIOS AUTORAIS

11.3 Caderno de aplicação

Exercício A · Modificação controlada

Troque o passo por três e detecte pixels sem cobertura na dimensão 8. Escreva uma função de posições que sempre inclua a última janela ancorada ao limite. Depois acrescente pesos menores na borda de cada patch e normalize pela soma desses pesos.

Comentário de solução

Com posições 0 e 3 e tamanho 4, o último índice coberto é 6; a última linha e coluna ficam descobertas. Incluir a posição dimensão-tamanho corrige a lacuna. Pesos que chegam exatamente a zero na borda podem deixar pixels externos sem contribuição; use uma política específica para limites ou um piso positivo. A normalização deve usar soma dos pesos, não quantidade de janelas.

Exercício B · Aplicação ampliada

Substitua a identidade por uma CNN e compare inferência integral com inferência por janelas usando margem de contexto. Recorte a borda da previsão de cada janela quando necessário e descreva qual região efetivamente contribui à reconstrução.

Critérios específicos de revisão

Confira cobertura completa e erro de reconstrução da identidade antes de atribuir costuras ao modelo.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 12 · TEORIA E MÉTODO

12. Checkpoint, esquema e reprodução de inferência

12.1 Fundamentos

Um checkpoint de inferência pode armazenar `state_dict` com os parâmetros da rede. A arquitetura e o pré-processamento precisam ser reconstruídos de forma compatível; pesos sozinhos não identificam a ordem de bandas nem a resolução. Um manifesto associado deve registrar versão, arquitetura, classes, normalização, divisões e métricas.

Salvar o objeto completo acopla o arquivo à estrutura Python e envolve mecanismos de serialização que exigem confiança na origem. Para o exercício, use `state_dict` e `torch.load` com `weights_only=True`. Isso reduz o escopo do carregamento, mas não transforma qualquer arquivo desconhecido em confiável. Não execute código anexado ou carregue modelos de origem não verificada em um ambiente de produção.

O laboratório salva pesos em um diretório temporário, recria a arquitetura e compara logits. O teste de igualdade dentro de tolerância verifica reconstituição dos parâmetros. O modelo precisa estar em eval para inferência consistente e a CPU recebe os pesos por `map_location`. Em uma entrega real, inclua um lote de referência e saída conhecida para detectar mudanças de esquema ou dependência.

Hipóteses e aprofundamento técnico

Reproduzir inferência exige mais do que um seed: arquitetura, versões, parâmetros de normalização, banda, dtype e modo da rede integram o contrato. Um `state_dict` inclui buffers como estatísticas de BatchNorm quando presentes, além de parâmetros aprendidos. A retomada de treino exige também época, estado do otimizador e configuração de sorteio; não confunda esse pacote com um checkpoint mínimo de inferência. Um arquivo de teste com tensor de entrada e logits esperados ajuda a detectar mudanças de comportamento entre versões. A tolerância deve refletir precisão numérica, sem esconder alterações de esquema.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
state_dict	Pesos e buffers	Exige arquitetura compatível
map_location	CPU no laboratório	Define dispositivo do carregamento
weights_only	Carregamento restrito	Não autoriza origem desconhecida
Manifesto	Bandas e normalização	Parte necessária da entrega

Conserve arquitetura, pesos, normalização e divisão de dados. Teste a reprodução em processo novo antes de publicar um resultado.

CAPÍTULO 12 · PROGRAMAÇÃO

12.2 Laboratório executável

Arquivo incorporado: capitulo_12.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
from tempfile import TemporaryDirectory
from pathlib import Path
import torch
from torch import nn
torch.manual_seed(32)
a=nn.Conv2d(3,1,1); a.eval(); x=torch.rand(1,3,4,4)
with TemporaryDirectory() as pasta:
    arq=Path(pasta)/'pesos.pt'
    torch.save(a.state_dict(),arq)
    b=nn.Conv2d(3,1,1)
    b.load_state_dict(torch.load(arq,map_location='cpu',weights_only=True))
    b.eval()
    with torch.inference_mode():
        ya,yb=a(x),b(x)
    assert torch.allclose(ya,yb)
    RESULTADO={'erro_max':float((ya-yb).abs().max()),
               'chaves':sorted(b.state_dict().keys())}
print(RESULTADO)
```

Resultado de referência calculado

```
{"chaves": ["bias", "weight"], "erro_max": 0.0}
```

Leitura do código

O diretório temporário contém apenas os pesos gerados pelo próprio exemplo. `load_state_dict` compara chaves e dimensões com uma arquitetura reconstruída. `map_location="cpu"` torna o dispositivo explícito e `weights_only` limita o carregamento aos elementos esperados. `eval` e `inference_mode` são aplicados antes da comparação, evitando diferenças de modo ou criação desnecessária de grafo.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

12.3 Caderno de aplicação

Exercício A · Modificação controlada

Troque a arquitetura de três para quatro canais e capture a incompatibilidade de `state_dict`. Crie um manifesto JSON com nomes de bandas, médias, desvios, pixel size, classes e SHA-256 do checkpoint. Inclua um teste que rejeite bandas fora de ordem.

Comentário de solução

Uma convolução com quatro canais espera peso de forma diferente; `load_state_dict` deve recusar o arquivo de três canais. O hash identifica o artefato, mas não comprova seu desempenho. A reprodução correta depende também de pré-processamento e esquema. A saída do laboratório tem erro máximo zero na comparação dos dois modelos reconstruídos.

Exercício B · Aplicação ampliada

Prepare um pacote de inferência que receba tensor, máscara e metadados; valide o contrato antes de executar. Entregue um relatório por cena, um teste de reconstrução em janelas e um checkpoint selecionado somente por validação.

Critérios específicos de revisão

Conserve arquitetura, pesos, normalização e divisão de dados. Teste a reprodução em processo novo antes de publicar um resultado.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

Referências e ambiente reproduzível

Fontes técnicas primárias para consultar assinaturas de API, modelos de dados e hipóteses de uso. A organização didática, os exercícios e os dados sintéticos são próprios deste volume. Documentação de software não substitui avaliação do desenho do estudo.

- <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>
- <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>
- <https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html>
- https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
- https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html
- <https://rasterio.readthedocs.io/en/stable/topics/windowed-rw.html>

Ambiente que executou os laboratórios

Python: 3.13.5; geopandas: 1.2.0; shapely: 2.1.2; scikit-learn: 1.7.1; numpy: 2.2.5; pandas: 3.0.6; torch: 2.8.0+cpu; rasterio: 1.4.3; PyMuPDF: 1.28.2

Instalação e extração

```
python -m venv .venv
# Windows: .venv/Scripts/python.exe
python -m pip install -r requirements.txt
python capitulo_01.py
```

O requirements.txt está incorporado ao PDF e enumera as dependências do volume. A instalação deve ocorrer no ambiente virtual escolhido. Para PyTorch CPU, use o índice oficial de wheels CPU conforme a documentação da plataforma. Não é necessário baixar dados ou pesos para executar os laboratórios.

Os scripts não dependem de execução prévia de capítulos e os arquivos de saída de demonstração usam diretórios temporários. Para adaptar a dados reais, substitua entradas com contrato equivalente e revise datas, suporte e critérios; não apenas os nomes dos arquivos. O anexo resultados_laboratorios.json permite conferir as saídas da edição.

Projeto integrador e avaliação

Treinar uma CNN pequena para segmentação binária em patches sintéticos; separar cenas inteiras, mascarar nodata, calcular IoU por cena e reconstruir uma imagem por janelas sobrepostas. Entregar pesos, esquema de bandas, parâmetros de normalização e um relatório que diferencie o teste didático de uma validação com sensores reais.

Entregáveis

- Relatório com pergunta, unidade de análise, período, fontes e hipótese do procedimento.
- Código organizado em etapas, com contrato de entrada e asserções coerentes com os capítulos.
- Tabela de verificação que confronte resultado principal com uma alternativa ou cálculo de referência.
- Artefatos reabertos em processo novo, acompanhados de manifesto de versões e parâmetros.
- Discussão das limitações específicas e dos dados adicionais necessários para aplicação real.

DIMENSÃO	CRITÉRIO	PONTOS
Escopo e suporte	Pergunta e unidades explícitas, dados identificados.	15
Programação	APIs corretas, fluxo reproduzível e tratamento de casos inválidos.	25
Avaliação	Controles específicos, baseline ou conservação verificável.	25
Interpretação	Separar resultado numérico, hipótese e conclusão territorial.	20
Entrega	Arquivos, documentação e reprodução por outro estudante.	15

Revisão final

Escolha dois capítulos que tenham alterado uma decisão do projeto e explique como. Demonstre um caso em que o fluxo deve rejeitar a entrada e outro em que a saída é matematicamente válida, mas inadequada à pergunta. Essa distinção é necessária para concluir a entrega.

O avaliador deve conseguir localizar cada fonte, executar o código e conferir uma saída sem reconstruir escolhas não documentadas. Registre erros encontrados e correções realizadas. Uma métrica elevada ou um mapa bem apresentado não compensa uma referência espacial, data, máscara ou divisão de dados incompatível.